

SDEV 2351

DAY 4: FLUXOR — ACTIONS AND REDUCERS

FROM PAPER DESIGN TO C# CODE

Session Objectives

1. Set up Fluxor in a Blazor application (NuGet, Program.cs, StoreInitializer)
2. Define state as immutable record types and actions as parameterized records
3. Implement reducers as pure static methods using the `[ReducerMethod]` attribute
4. Wire a Blazor component to a Fluxor store using `FluxorComponent`, `IState<T>`, and `IDispatcher`
5. Write a unit test for a Fluxor reducer

Today's Schedule

PHASE	TIME	WHAT'S HAPPENING
Connecting	10 min	C# Events bridge + pair recall of the Flux cycle
Exploring	25 min	Fluxor setup, records, actions, reducers, component wiring
Practicing	65 min	Counter extensions, todo list store, reducer testing
Assessing	10 min	Exit ticket — writing a reducer test in AAA format

PHASE 1 · 10 MINUTES

CONNECTING

From C# events to Fluxor dispatch

C# Events to Fluxor

Day 3's paper-based design becomes real C# code today.

C# EVENTS (KNOWN)

Delegate = publisher-subscriber

EventArgs = data carrier

Event handler = mutates state

FLUXOR (NEW)

IDispatcher = publisher

Action record = data carrier

Reducer = returns NEW state

The ONE Big Difference

Event handlers mutate. Reducers replace.

EVENT HANDLER

`count++;` — mutates in place

REDUCER

`return state with { Count = state.Count + 1 };`
— returns new state

This is the most important distinction for today.

PHASE 2 · 25 MINUTES

EXPLORING

Fluxor setup, state records, actions, reducers, and component wiring

Fluxor Setup Checklist

1. **NuGet:** Install `Fluxor.Blazor.Web`
2. **Program.cs:** `builder.Services.AddFluxor(o => o.ScanAssemblies(...))` (+ [ReduxDevTools package ref](#))
3. **_Imports.razor:** Add Fluxor `@using` statements
4. **App.razor:** Add `<StoreInitializer />` component

Once configured, Fluxor automatically discovers your state, actions, and reducers via assembly scanning.

Records vs Classes

C# **records** are immutable-friendly alternatives to classes:

FEATURE	CLASS	RECORD
Equality	Reference (same object)	Value (same data)
Properties	<code>get; set;</code>	<code>get; init;</code>
Copy with changes	Manual clone	<code>with</code> expression

```
record CounterState(int Count);  
var newState = oldState with { Count = oldState.Count + 1 };
```

We use records for Fluxor state because state is compared **by value**.

State, Actions, and Reducers

State — immutable `record` with `init` properties

Actions — `record` types describing what happened

- Parameterless for signals: `record IncrementCounterAction;`
- With data: `record SetCounterAction(int NewCount);`

Reducers — static pure functions with `[ReducerMethod]`

- Take current state + action, return new state
- No side effects, no async, no DI

Component Wiring

```
@inherits FluxorComponent
@Inject IState<CounterState> State
@Inject IDispatcher Dispatcher

<p>Count: @State.Value.Count</p>
<button @onclick="() => Dispatcher.Dispatch(
    new IncrementCounterAction())">+1</button>
```

- `@inherits FluxorComponent` — auto-subscribes to state changes
- `IState<T>` — reads current state via `State.Value`
- `IDispatcher` — sends actions to the store

Full code: [see reference-sheet.md](#) or your IDE

Counter Store — Demo Recap

Files created during the live demo:

FILE	PURPOSE
<code>CounterState.cs</code>	<code>record</code> with <code>Count</code> property
<code>CounterActions.cs</code>	Increment, Decrement, Set actions
<code>CounterReducers.cs</code>	Three <code>[ReducerMethod]</code> static methods
<code>CounterFeature.cs</code>	<code>Feature<CounterState></code> with <code>GetInitialState()</code>
<code>Counter.razor</code>	Component wiring with FluxorComponent

Open **Redux DevTools** to see the action log and state snapshots after each dispatch.

PHASE 3 · 65 MINUTES

PRACTICING

Hands-on Fluxor store building + reducer testing

Practice 1: Counter Extensions (Guided, 10 min)

Starter provides: Counter store files from the demo (state, feature, one reducer)

You add:

- `ResetCounterAction` record
- Reducer method for reset
- Wire the Reset button in `Counter.razor`
- Conditional message when count is zero

Practice 2: Todo List Store (Independent, 27 min)

Starter provides: `TodoItem` model, empty `TodoState` record, component UI with form and list. `ToggleTodoAction` and its reducer are pre-built.

You write:

- `AddTodoAction(string Title)` and `RemoveTodoAction(int Id)` actions
- Both reducer methods
- `TodoFeature` class
- Component wiring (`@inherits`, `@inject`, `dispatch`)

Success threshold: Complete Add action + its reducer + `TodoFeature` + component wiring for Add. That proves you understand the Fluxor cycle.

Your First Reducer Test (Guided, 10 min)

Reducers are **pure functions** — test them without Fluxor running:

```
[Fact]
public void ReduceIncrementCounterAction_WithCountOf5_ReturnsCountOf6()
{
    var state = new CounterState { Count = 5 };
    var action = new IncrementCounterAction();
    var result = CounterReducers.ReduceIncrementCounterAction(state, action);
    Assert.Equal(6, result.Count);
}
```

Key insight: Just call the static method with inputs and check the output. No mocking, no setup.

Full code: [see reference-sheet.md](#) or your IDE

Practice 3: Cart Refactor (Take-Home, Optional)

Starter provides: 80% of store files (CartState, CartFeature, AddToCart + UpdateQuantity pre-built)

You complete:

- `RemoveFromCartAction` and its reducer
- Component wiring for both Products and Cart pages
- Navigate between pages to verify state persists

This exercise reinforces today's concepts. Complete before Day 5.

Troubleshooting Tips

SYMPTOM	LIKELY CAUSE
Component doesn't update after dispatch	Missing <code>@inherits FluxorComponent</code>
State resets on navigation	Missing <code>Feature<T></code> class or <code>GetInitialState()</code>
Build error on state assignment	Using <code>set</code> instead of <code>init</code> (mutating state)
Reducer not called	Missing <code>[ReducerMethod]</code> attribute
Redux DevTools shows nothing	Missing the <code>Fluxor.Blazor.Web.ReduxDevTools</code> package reference

PHASE 4 · 10 MINUTES

ASSESSING

Summary

Today you learned to:

1. **Set up Fluxor** in a Blazor app with NuGet, Program.cs, and StoreInitializer
2. **Define state** as immutable records and **actions** as parameterized records
3. **Implement reducers** as pure static methods with `[ReducerMethod]`
4. **Wire components** to the store using `FluxorComponent`, `IState<T>`, and `IDispatcher`
5. **Unit test reducers** directly — pure functions need no infrastructure setup

Exit Ticket

Write the three AAA sections for testing a reducer that removes a todo item from a list of 3 items.

Use pseudocode or real C#:

- **Arrange:** Create a state with 3 items and a `RemoveTodoAction`
- **Act:** Call the reducer method directly
- **Assert:** Verify the returned state has 2 items

What's Next

Day 5: Fluxor Effects and Async Operations

- Today's reducers are synchronous and pure
- But what if you need to load data from an API?
- **Reducers can't do async work** — that's what effects are for
- Tomorrow: effects that call services and dispatch success/failure actions

No pre-class reading assigned.